

Here is a **pre-deployment, failure-oriented review** of your installer. I'm going to treat this like production-critical infrastructure (because it is): anything that touches partition tables, bootloaders, and auth must assume adversarial conditions and operator error.

I'll go through your requested areas first, then highlight **systemic risks and design-level issues**.

High-Level Assessment

Strengths

- Clean separation: backend vs TUI is well-structured.
- Logical install flow mirrors LFS expectations.
- Good use of UUID in fstab (mostly).
- Chroot abstraction is simple and reusable.
- Installer is readable and maintainable (~1.2k LOC, good).

Critical Concerns

- **Destructive operations lack guardrails**
- **Shell execution model is unsafe (shell=True everywhere)**
- **Mount / chroot lifecycle is not failure-safe**
- **Password handling is insecure**
- **Disk targeting is fragile**
- **No rollback or partial failure containment**

If you ran this today on real hardware, I would expect:

- Occasional **wrong-disk wipes**
- **Broken boot installs** (especially EFI edge cases)
- **Dangling mounts / corrupted installs after failure**
- Potential **credential leakage via process table**

1. Disk Partitioning Safety ⚠️ (High Risk)

Issues

✗ No strong confirmation barrier

You warn once:

```
if not self.yes_no(row, "Proceed with partitioning?", default=False):
```

That is **not sufficient** for destructive ops.

✗ No disk identity validation

You rely on:

```
choice = self.prompt(...)
```

But:

- No re-display of disk **serial / WWN**
- No check against **root disk of live environment**
- No protection against wiping installer media

✗ Installer media exclusion is incomplete

You exclude:

```
if name.startswith(("loop", "ram", "sr", "zram"))
```

But:

- USB install media will appear as /dev/sdX → **you will wipe it**
- No check for mounted root (/) backing device

✗ Partition creation assumes timing success

Immediately after parted, you call:

```
_partition_paths(...)
```

No:

- udevadm settle
- re-read partition table (partprobe)

→ Race condition risk.

Recommendations

Minimum safety upgrades:

- Require typed confirmation:

Type the disk path to confirm: /dev/sda

- Exclude active root device:

```
findmnt -n -o SOURCE /
```

- Add:

```
lsblk -no PKNAME $(findmnt -n -o SOURCE /)
```

- Force sync after partitioning:

```
_run("partprobe")
```

```
_run("udevadm settle")
```

2. Fstab Generation ⚠ (Moderate Risk)

Issues

✗ Hardcoded filesystem assumptions

ext4, vfat

This is fine *only because you control formatting*, but:

- No mount options tuning:
 - Missing noatime, errors=remount-ro, etc.

✗ Weak fallback to device path

else:

```
lines.append(f"{partitions['root']} ...")
```

If UUID fails → fragile boot.

Recommendations

- Fail hard if UUID is missing:

if not root_uuid:

```
raise RuntimeError("Failed to resolve UUID")
```

- Improve mount options:

ext4: defaults,noatime

vfat: umask=0077

3. Chroot Hook Execution (High Risk)

Issues

Mount operations are unchecked

```
subprocess.run(cmd, shell=True, capture_output=True)
```

- No check=True
- Silent failures → catastrophic later

No mount state tracking

You assume mounts succeed and blindly unmount:

```
umount {target}/{sub}
```

→ Can:

- Fail silently
- Leave leaked mounts
- Break subsequent runs

No failure isolation

If a hook:

- hangs
- partially executes

→ installer continues

Recommendations

- Track mounts explicitly:

```
mounted = []
```

- Use check=True for all mounts
- Add timeout to chroot:

```
subprocess.run(..., timeout=...)
```

- Consider systemd-nspawn instead of raw chroot

4. User Creation Security (Critical)

Issues

Plaintext passwords via shell

```
echo 'user:password' | chpasswd
```

Problems:

- Visible in process table (ps)
- Shell injection risk if password contains '

No password policy

- Empty password prevented (good)
- But no:
 - length enforcement
 - complexity
 - entropy check

Recommendations

Use stdin instead:

```
subprocess.run(
    ["chroot", target, "chpasswd"],
    input=f"{username}:{password}",
    text=True,
    check=True
)
```

Avoid shell entirely.

5. GRUB Installation ⚠ (Moderate–High Risk)

Issues

✗ EFI detection too simplistic

```
return os.path.isdir("/sys/firmware/efi")
```

Fails in:

- chrooted installs
- some VM configs

✗ Missing required packages validation

You assume:

- grub-install
- efibootmgr

exist inside target.

✗ No fallback or diagnostics

If GRUB fails → user gets vague error.

Recommendations

- Validate tools inside chroot:

which grub-install

- Log full stdout/stderr visibly
- For EFI:
 - ensure /boot/efi is FAT32
 - verify mount before install

6. TUI Robustness ⚠ (Moderate Risk)

Issues

✗ No resize handling

No KEY_RESIZE handling.

✗ Input validation weak

- Hostname not validated
- Username not validated

✗ Package selection not implemented

Simple toggle — for now just accept defaults

→ misleading UI

Recommendations

- Add validation regex:
 - hostname: RFC 1123
 - username: POSIX compliant
- Handle:

if key == curses.KEY_RESIZE:

7. General Safety 🚨 (Critical)

Major systemic issues

✗ 1. shell=True everywhere

You do:

```
_run(f"parted -s {disk_path} mklabel gpt")
```

This is:

- injection-prone
- brittle
- unnecessary

Even if input is “trusted”, **this is installer code running as root.**

✘ 2. No transactional model

Installer is **non-atomic**:

If failure occurs mid-install:

- Disk is wiped
- System half-installed
- No recovery path

✘ 3. No logging strategy

You rely on:

`capture_output=True`

But:

- Errors are not persisted
- No install log for debugging

✘ 4. No dry-run mode

You need:

`--dry-run`

Especially before first deployment.



Additional Observations

Package system

- Clean abstraction
- But no dependency validation before install

Hooks system

- Flexible
- But executing arbitrary build.sh inside chroot is **high trust risk**

Mount lifecycle

- Shared between modules → duplication risk



Priority Fix List (Order Matters)



Must fix before ANY test

1. Remove shell=True everywhere
2. Fix password handling (stdin-based)
3. Add disk confirmation (typed + root disk exclusion)
4. Add udevadm settle after partitioning
5. Make mount operations strict (check=True)
6. Fail hard on UUID resolution failure

Should fix before VM testing

7. Add logging (/var/log/forge-installer.log)
8. Validate GRUB dependencies
9. Add mount tracking + cleanup guarantees
10. Add input validation (username/hostname)

Before real hardware

11. Add dry-run mode
12. Add install summary + export
13. Add retry/recovery strategy

Final Verdict

Architecturally solid, operationally unsafe (current state).

You've built a **clean LFS-style installer framework**, but it lacks the defensive engineering required for something that:

- wipes disks
- installs bootloaders
- provisions credentials

Right now, it behaves like a **trusted lab tool**, not a **production installer**.

If you want, I can:

- Refactor one module (e.g., disks.py) into a **production-safe version**
- Or design a **transactional install model** (with rollback + journaling)